

ORCHES: Orchestrated Test-Time-Compute-based LLM Reasoning on Collaborative GPU-PIM Heterogeneous System

Sixu Li*

Georgia Institute of Technology
Atlanta, USA
sli941@gatech.edu

Yuzhou Chen*

Georgia Institute of Technology
Atlanta, USA
eiclab.gatech@gmail.com

Chaojian Li

Georgia Institute of Technology
Atlanta, USA
cli851@gatech.edu

Yonggan Fu

Georgia Institute of Technology
Atlanta, USA
yfu314@gatech.edu

Zheng Wang

Georgia Institute of Technology
Atlanta, USA
zwang3478@gatech.edu

Zhongzhi Yu

Georgia Institute of Technology
Atlanta, USA
zyu401@gatech.edu

Haoran You

Georgia Institute of Technology
Atlanta, USA
hyou37@gatech.edu

Zhifan Ye

Georgia Institute of Technology
Atlanta, USA
zye327@gatech.edu

Wei Zhou

Georgia Institute of Technology
Atlanta, USA
wzhou322@gatech.edu

Yongan Zhang

Georgia Institute of Technology
Atlanta, USA
yzhang919@gatech.edu

Yingyan (Celine) Lin

Georgia Institute of Technology
Atlanta, USA
celine.lin@gatech.edu

Abstract

Recent breakthroughs in AI reasoning, enabled by test-time compute (TTC) on compact large language models (LLMs), offer great potential for edge devices to effectively execute complex reasoning tasks. However, the intricate inference pipelines associated with TTC pose new efficiency bottlenecks, limiting achievable latency and hindering widespread adoption. Through an in-depth analysis, we identify three key barriers: (1) variable parallelism, characterized by inference-dependent dynamic control flows and varying batch sizes, complicating workload scheduling; (2) branch dependencies, hindering efficient pipelining across sequential reasoning steps; and (3) branch pruning, causing memory fragmentation and irregular data access patterns. Motivated by the memory-bound nature of LLMs and Processing-in-Memory (PIM)’s capability to reduce data movement, we propose ORCHES, a novel GPU-PIM collaborative system specifically designed to address these barriers. ORCHES integrates three key innovations: (1) adaptive workload assignment, dynamically balancing workloads between GPU and PIM units to maximize parallelism despite unpredictable branching; (2) branch-aware pipelining, leveraging speculative execution to substantially reduce inter-step pipeline stalls; and (3) fragmentation-aware memory structuring, enhancing data locality and access efficiency through coordinated caching and optimized memory layout reorganization. Experimental results demonstrate that ORCHES

achieves average speedups of 4.16× and 3.10× over state-of-the-art (SOTA) GPU implementations for text-based and vision-based reasoning tasks, respectively, without any loss in the accuracy of the original reasoning pipeline.

CCS Concepts

• **Computer systems organization** → **Heterogeneous (hybrid) systems**; • **Hardware** → **Application specific processors**; • **Computing methodologies** → **Artificial intelligence**.

Keywords

Processing-in-Memory, Heterogeneous Computing, Hardware Acceleration, Large Language Models

ACM Reference Format:

Sixu Li, Yuzhou Chen, Chaojian Li, Yonggan Fu, Zheng Wang, Zhongzhi Yu, Haoran You, Zhifan Ye, Wei Zhou, Yongan Zhang, and Yingyan (Celine) Lin. 2025. ORCHES: Orchestrated Test-Time-Compute-based LLM Reasoning on Collaborative GPU-PIM Heterogeneous System. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO ’25)*, October 18–22, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3725843.3756039>

1 Introduction

Recent breakthroughs in AI reasoning, enabled by large language models (LLMs), promise transformative real-world applications—from assisting coding tasks [29] to multi-hop question answering [6] and advanced 3D understanding [7, 36]. However, deploying these reasoning capabilities on everyday edge devices remains challenging due to resource constraints and immense computational demands.

*Sixu Li and Yuzhou Chen contributed equally to this work.



This work is licensed under a Creative Commons Attribution 4.0 International License. *MICRO ’25, Seoul, Republic of Korea*

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1573-0/25/10

<https://doi.org/10.1145/3725843.3756039>

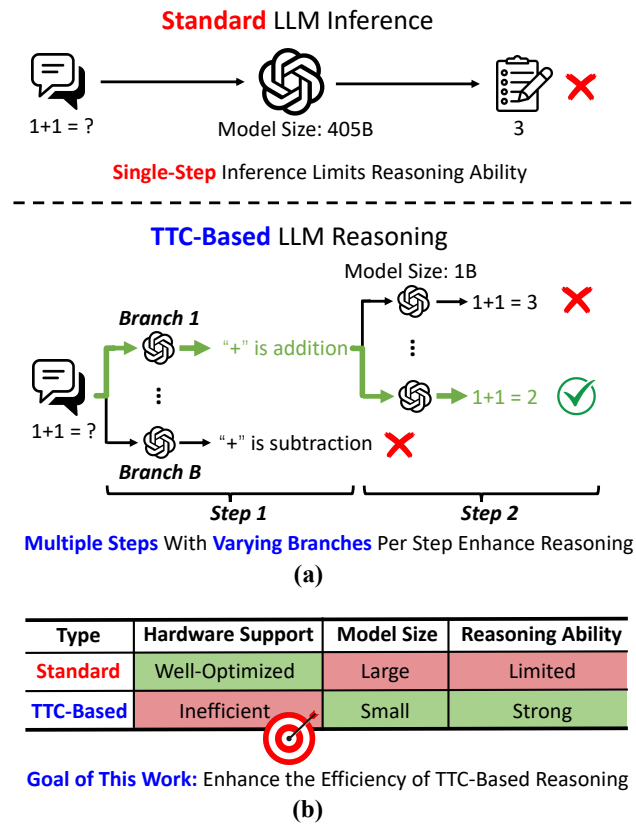


Figure 1: (a) Illustrative comparison between standard single-step LLM inference, which limits reasoning despite large models, and TTC-based multi-step reasoning, which enhances reasoning by increasing inference “width” (multiple branches per step) and “depth” (multiple sequential reasoning steps), enabling smaller models to outperform larger ones. (b) Summary of the trade-offs: unlike standard inference, TTC-based reasoning achieves stronger reasoning capability with smaller models but currently suffers from suboptimal hardware utilization.

For instance, solving a single problem from the widely-used reasoning benchmark MATH500 [8] requires around 10 minutes even on edge GPUs [23], severely limiting practicality.

To bridge the gap between the capabilities of large-scale reasoning models and the limited resources available on practical platforms, Test-Time Compute (TTC) has emerged as a highly promising paradigm [28]. Rather than directly performing a single-step inference, TTC decomposes each reasoning task into multiple sequential sub-tasks. At every step, the model generates multiple candidate solutions (“branches”), evaluates these branches using a learned verification function, such as a Process Reward Model (PRM), and selects the most promising ones to proceed to the next step. This approach effectively enhances model performance, allowing relatively compact LLMs, e.g., models with only around 1B

parameters, to surpass much larger models (405B+ parameters) on challenging reasoning benchmarks [18].

Despite the aforementioned promise, the benefits of TTC introduce unique computational challenges that current hardware acceleration methods cannot effectively address. As analyzed in Sec. 2.2, general LLM decoding workloads tend to be memory-bound [9, 25], especially in edge scenarios, where Processing-in-Memory (PIM) architectures have emerged as promising accelerators, mitigating memory bottlenecks by reducing data movement [5, 17, 27]. However, existing PIM-based solutions have primarily focused on single-step LLM inference [9, 25]. We note that both current GPU and PIM acceleration strategies fall short in addressing the distinctive computational challenges posed by TTC-powered reasoning workloads due to the challenges summarized below and illustrated in Fig. 2: **Challenge 1 (C1) – Variable Parallelism Complicating Workload Scheduling:** Unlike conventional LLM decoding at edge, which is uniformly memory-bound, TTC reasoning introduces a mixture of compute- and memory-bound behavior. This stems from the distinct roles of policy models (decoding) and PRMs (prefilling), as well as the presence of both shared and unique KV caches across candidates. Shared KV usage can make operations compute-bound due to higher parallelism, while unique KV access remains memory-bound. In addition, as the reasoning progresses, the ratio of shared-to-unique KV cache dynamically shifts, causing compute patterns to evolve. This heterogeneity complicates scheduling and mapping. **Challenge 2 (C2) – Branch Dependencies Hindering Pipeline Execution:** In standard LLM decoding, each token generation depends sequentially on previous tokens due to the auto-regressive nature. TTC-powered LLM reasoning introduces an additional layer of sequential dependency across reasoning steps: candidate generation at each step must wait for verification results from the previous step, and vice versa. This strictly enforced inter-step execution order significantly limits the effectiveness of traditional pipelining techniques. **Challenge 3 (C3) – Branch Pruning Inducing Memory Fragmentation:** During TTC reasoning, unselected branches are removed from memory as the process progresses, resulting in irregular memory access patterns and runtime fragmentation. This irregular behavior reduces the utilization efficiency of vanilla PIM accelerators and limits the overall achievable system energy efficiency.

In response to these unique efficiency challenges posed by TTC-powered LLM reasoning, we propose a novel GPU-PIM collaborative system, ORCHES, specifically designed to address these challenges. ORCHES integrates three new techniques, each explicitly targeting one of the identified barriers: **Technique 1 (T1) – Adaptive Assignment Enhancing Parallelism:** To leverage parallelism opportunities in LLM reasoning fully, we propose a workload assignment strategy for collaborative GPU-PIM execution. This includes (1) offline computation partitioning between GPU and PIM modules based on anticipated batch sizes and reasoning branches, and (2) online runtime scheduling to dynamically compensate for workload imbalance between GPU and PIM modules as reasoning steps progress. **Technique 2 (T2) – Branch Prediction Facilitating Pipelining:** To improve pipelining across adjacent reasoning steps, we introduce a branch prediction mechanism tailored for LLM reasoning, inspired by CPU branch prediction designs. Specifically, we develop a lightweight predictor that forecasts

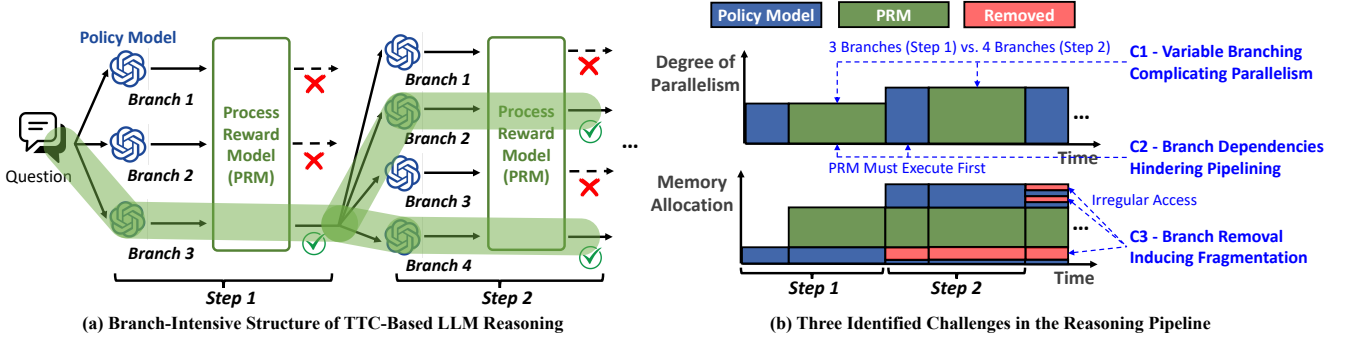


Figure 2: An overview of how (a) the unique branch-intensive structure of TTC-based LLM reasoning leads to (b) the three identified challenges in the reasoning pipeline. We use 3 and 4 branches in this figure as illustrative examples to help visualize the dynamic branching behavior; the actual number of branches may vary depending on the task or configuration.

which branches are likely to be selected in the next step, enabling the system to generate their outputs in parallel with the verification process of the current step. This prediction-guided execution allows overlapping output generation and branch selection, improving pipeline utilization and reducing inter-step latency. **Technique 3 (T3) – Memory Structuring Alleviating Fragmentation:** To address irregular memory access patterns due to fragmentation, we propose a memory structuring strategy combining a dedicated cache for frequently accessed branches, lightweight memory reorganization for improved contiguity, and a controller-side buffer optimizing GPU access. This strategy enhances data locality and mitigates fragmentation in TTC-powered workloads.

Evaluating ORCHES on both text-based and vision-based reasoning tasks demonstrates substantial speedups of 4.16× and 3.10× over state-of-the-art (SOTA) GPU implementations, respectively, all while fully preserving original inference accuracy.

2 Background

2.1 Large Language Models (LLMs)

Recent advances in LLMs have been driven by the decoder-only Transformer architecture, which underpins many SOTA models, such as OpenAI’s GPT-4 [24] and Meta’s LLaMA series [31–33]. These models are typically built by stacking multiple identical decoder blocks, each sharing the same architecture but with distinct learned parameters.

Decoder blocks comprise three sequential components: **1) Linear operators project** input tokens into query (Q), key (K), and value (V) vectors for attention, with K and V stored in the KV cache for efficient autoregressive generation; **2) Attention operators** compute scores by comparing each token’s Q with cached K , normalize them via softmax, and produce a weighted sum of cached V ; **3) Linear operators in feed-forward networks (FFNs)** process each token with two linear transformations and a non-linear activation, combining the output with the input via a residual connection.

During generation, decoding-only Transformers operate in two stages: **prefilling** and **decoding**. The **prefilling** stage occurs once at the beginning, when the full input prompt (e.g., user instruction or context) is available. In this stage, all prompt tokens are processed in parallel to compute their key (K) and value (V) vectors, which are

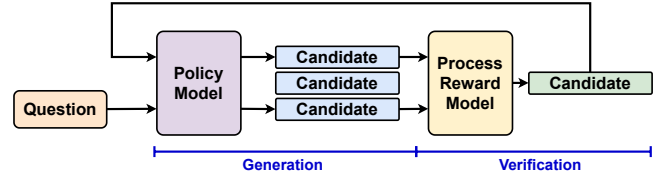


Figure 3: Block diagram of the TTC-based LLM reasoning pipeline: the generation phase and the verification phase.

stored in the KV cache for reuse. The **decoding** stage follows and proceeds autoregressively. At each step, a new token is generated based on all previously generated tokens, using the cached K/V values to avoid redundant computation. As token generation is inherently sequential, this stage becomes the primary efficiency bottleneck, particularly for long outputs or interactive use cases.

2.2 Test-Time Compute (TTC) based Reasoning

TTC has emerged as a promising direction to enhance LLMs’ performance on complex tasks requiring multi-step reasoning. Unlike conventional single-step generation, TTC introduces a structured, iterative reasoning process that allows the model to decompose tasks into intermediate steps and progressively refine its output.

As shown in Fig. 3, to solve an input question—referred to as a **request** in this paper—a typical TTC pipeline [18, 36] adopts an iterative generation pattern to break down complex problems into sequential steps. Each step consists of a **generation phase**, which produces a set of candidate outputs conditioned on the current prompt, followed by a **verification phase** that selects the most promising candidates. The selected candidates are appended to the prompt and used to proceed to the next step, while unselected candidates are discarded. This iterative procedure continues until the input question is solved. The overall procedure can be viewed as a form of tree search, where nodes represent candidate outputs and edges correspond to reasoning transitions. The width (number of candidates per step) and depth (number of reasoning steps) are task-specific and typically determined empirically. To support this iterative reasoning, TTC systems typically consist of two main components: a **policy model**, which drives the **generation phase**

by producing candidate outputs (corresponds to **decoding** in standard LLM inference), and a **process reward model (PRM)**, which performs the **verification phase** by selecting the most promising candidate (essentially a **prefilling** task), as shown in Fig. 3.

Together, this combination of multi-branch generation and selective verification forms a structured but asymmetric computation pattern, introducing a unique test-time workload that diverges from standard LLM inference.

2.3 Processing-in-Memory (PIM)

TTC introduces highly irregular workloads with small batch sizes, dynamic branching, and sequential inter-step dependencies. These result in a mix of compute- and memory-bound behavior across different reasoning stages, diverging significantly from the regular, high-throughput workloads GPUs are optimized for. Although GPUs provide strong computing capabilities, their effective memory bandwidth is constrained by architectural limitations: Modern memory modules multiplex access across banks and channels, exposing only a fraction of internal bandwidth at any given time [12], which can lead to under-utilization, especially for memory-bound operations like attention with batch size 1.

PIM architectures address this issue by integrating lightweight compute units directly into memory banks, reducing data movement and increasing effective bandwidth [5, 17, 27, 34, 41]. While prior works [9, 25] focus on server-level LLM inference, they highlight a key principle: Operators with low compute-to-memory ratios are more sensitive to bandwidth constraints and can benefit from memory-centric architectures. In TTC settings, these memory bottlenecks are even more pronounced, and are further exacerbated by inter-stage dependencies and memory sparsity caused by branch pruning. These insights motivate a heterogeneous GPU-PIM system tailored to the unique workload characteristics of TTC-based LLM reasoning.

3 Identified New Patterns and Challenges

3.1 Challenge 1: Variable Parallelism Complicates Workload Scheduling

In regular LLM decoding, both linear and attention operators are memory-bound, with low data reuse and parallelism. In contrast, TTC presents a more complex workload pattern: (1) Since policy models involve a decoding process and PRMs involve a prefilling process, under an edge setting (i.e., solving one request at a time), the linear operators in each can be memory-bound or compute-bound, respectively; (2) There exist both shared KV caches among all candidates, due to shared reasoning trajectories in beam search, and unique KV caches specific to each candidate. The former may result in a compute-bound scenario due to increased opportunities for parallel computation, while the latter corresponds to a memory-bound scenario. Consequently, this variation in parallelism complicates workload scheduling and mapping across platforms.

In addition, the ratio of shared-to-unique KV caches may dynamically evolve as the search process deepens, leading to dynamic compute behavior and further complicating workload scheduling and mapping throughout the search process. These new workload patterns are elaborated below.

3.1.1 New Workload Patterns of Different Operators. To analyze the compute- or memory-bound scenarios of operators in TTC-based LLM reasoning workloads, we follow the definition from prior work [39] to derive the arithmetic intensity, i.e., the ratio of FLOPs to bytes accessed, as a function of batch size W , defined as the number of input tokens to the LLM that can be processed in parallel. If an operator's arithmetic intensity exceeds a device-specific threshold, the workload becomes compute-bound and demonstrates good data reuse, favoring compute-centric platforms with high parallelism such as GPUs. Conversely, a low arithmetic intensity indicates a memory-bound workload, which is better suited to memory-centric architectures such as PIM. We analyze the workload patterns for linear and attention operators as follows.

Arithmetic intensity of linear operators. As shown in Fig. 4, the arithmetic intensity of the linear operator generally increases with batch size W . In PRMs, which correspond to the verification phase, the workload primarily involves a prefilling process with a large W , often equal to the input sequence length L (typically >100) in common use cases [8, 36]. This results in high arithmetic intensity and compute-bound workloads, even for a single request, making GPUs the ideal choice. In contrast, policy models follow a token-by-token decoding process, characterized by a small W on edge devices. For example, in text-based TTC pipelines, the number of candidates can be as low as 4 [18], and even down to 2 in vision-based TTC pipelines [2, 36], leading to memory-bound workloads that are better suited for PIM architectures.

Arithmetic intensity of attention operators. A similar trend holds for the attention operator. In traditional setups, where each request maintains a unique KV cache, the workload is consistently memory-bound, making it well-suited for PIM architectures. However, with the introduction of shared KV caches across candidates in reasoning workloads (as discussed in Sec. 2.2), the workload can become compute-bound if the number of candidates is sufficiently large. Since both shared and unique KV caches coexist in TTC, a new parallelization strategy is required to efficiently support both associated with the same operator.

Identified Challenge 1A

Unlike regular LLM decoding, where all operators are consistently memory-bound, edge reasoning workloads may exhibit a mix of compute-bound and memory-bound behaviors across different operators.

3.1.2 Dynamically Evolving Workload Patterns During Search. We further analyze the impact of dynamically evolving workload patterns caused by the increasing ratio of shared to unique KV caches as the search process progresses, leading to dynamically evolving compute behavior. Specifically, as mentioned in Sec. 2.2, as the search process progresses, the selected candidates are appended to the original prompt to form a new prompt and serve as the inputs for the next step. As a result, the *shared* KV cache increases as the search deepens, as shown in Fig. 5. In contrast, the *unique* KV cache for each candidate is always cleared when moving to the next step. In other words, since the unique context depends only on the current candidate, it is reset to zero at the start of each new candidate. As a result, the workload associated with shared

KV caches increases as the search deepens, while the workload for the unique context remains relatively unchanged, causing an imbalance in compute utilization over time. Since the workloads from both shared and unique KV caches across all candidates must be aggregated to proceed with the search process, this imbalance introduces additional delays due to the need for synchronization.

Identified Challenge 1B

The *shared* KV cache associated with the reasoning workload grows larger as the search progresses, whereas the *unique* KV cache workload remains relatively constant. This discrepancy leads to an imbalance and introduces synchronization overhead.

3.2 Challenge 2: Branch Dependencies Hinder Pipeline Execution

As discussed in Sec. 2.2, the verification phase primarily consists of prefilling, which is parallelizable and generally achieves good runtime on existing GPUs. In contrast, the generation phase is inherently sequential, making it slower on GPUs and better suited for PIM acceleration. However, for certain workloads, the verification phase can take as long as—or even longer than—the generation phase on GPUs, as shown in Fig. 6(a). This can be attributed to two key factors: (1) verification may require a larger model to ensure sufficient accuracy, which significantly increases computational cost; and (2) a large number of candidates may need to be verified, further compounding the runtime.

To provide a deeper analysis, Fig. 6(b) presents profiling results comparing decoding speed (the main workload of generation) and prefilling speed (the main workload of verification). We observe that, when using models of the same size, prefilling can be significantly faster than decoding—up to 50× in some cases. However, when the verification model is much larger, this speed advantage diminishes. For example, the prefilling speed of an 8B model is only about 10× faster than the decoding speed of a 1B model. In such scenarios, verification may become the bottleneck. Moreover, since verification (prefilling) benefits from GPU acceleration while generation (decoding) favors PIM, any stall in the verification phase can lead to under-utilization of the PIM module, negatively impacting

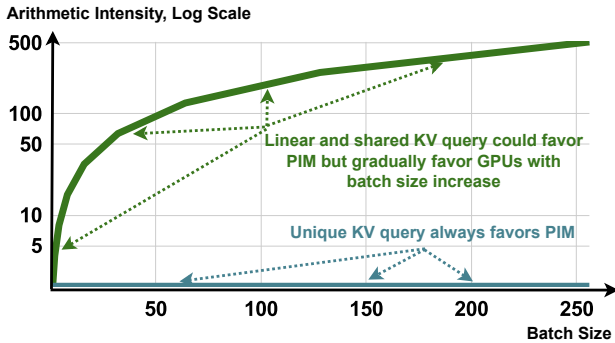


Figure 4: Illustration of the arithmetic intensity of linear and attention operators in TTC-based LLM reasoning workloads.

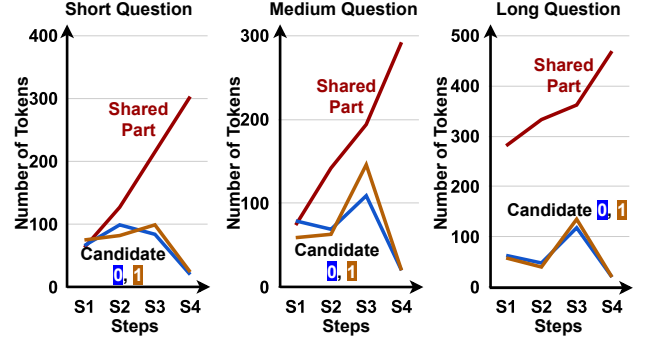


Figure 5: An example of the number of tokens in the *shared* KV cache and the *unique* KV cache during the reasoning process. The example is from the MATHVista [19] dataset using the SOTA TTC-based vision LLM reasoning model [36].

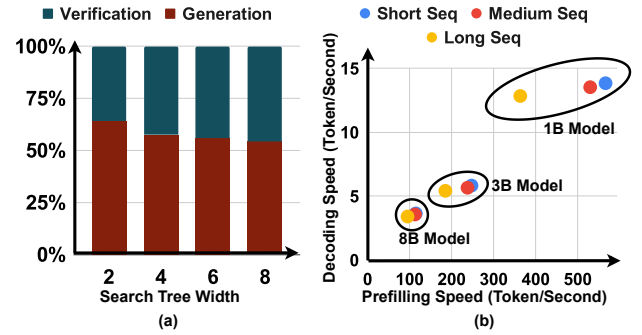


Figure 6: (a) An example of the verification and generation time breakdown in a SOTA TTC pipeline [18], where the policy model for generation is a 1B model and the PRM model for verification is an 8B model, and (b) the prefilling and decoding speed on an edge device [23] with different model sizes and sequence lengths.

overall system performance. Conversely, the verification phase also depends on the generation phase, as candidate content must be generated before it can be verified. Therefore, in scenarios where generation is offloaded to PIM, the GPU may become under-utilized due to this dependency, as it must wait for generation to complete before initiating verification.

Identified Challenge 2

Verification and generation phases in the reasoning workload exhibit data dependencies on each other and require different compute resources, which can lead to mutual bottlenecks, resulting in under-utilization of the GPU-PIM system.

3.3 Challenge 3: Branch Pruning Induces Memory Fragmentation

Unlike regular LLM edge inference, where the request consists of a single continuous sequence, TTC-based reasoning processes,

selects, and discards a set of candidates. Discarded candidates are never reused, corresponding to a branch pruning process. This can lead to nontrivial memory fragmentation, which is unfavorable for PIM devices due to the increased memory access overhead associated with indexing the selected candidate. The problem is further exacerbated by the variability in candidate lengths: our experiments on the MATH [8] dataset using a SOTA text-based TTC pipeline [18] show that the number of tokens per candidate can range from 10 to 1000. To achieve high memory utilization and avoid fragmentation, this sparsity must be carefully managed.

Identified Challenge 3

Branch pruning in TTC causes memory fragmentation, which is unfavorable for PIM devices due to additional memory access overhead and thus requires proper handling.

4 The Proposed ORCHES Framework

4.1 Hardware Architecture Overview

To address the unique challenges of accelerating TTC-based LLM reasoning, as discussed in Sec. 3, we develop the proposed ORCHES system, of which the hardware architecture is shown in Fig. 7. In particular, the architecture consists of the following three main components, each implemented as a separate silicon die: **Host GPU** refers to the conventional GPU typically used in edge devices [23]. While it offers ample computing resources, it is constrained by limited memory bandwidth. This module primarily handles compute-intensive tasks, such as prefilling and executing operators with large batch sizes during decoding. **Memory Controller Die** serves as the interface between the host GPU and the memory dies. Beyond standard memory read/write operations, it performs data aggregation across memory channels, as also explored in prior work [9, 25]. The aggregation in the controller die is supported by the Accum Units (implemented as parallel adders) and the Softmax Units (implemented using fixed-function pipelined datapath). Specifically, it is responsible for (1) accumulation operations and (2) statistical transformations such as Softmax and normalization. These operations are delegated to the controller due to their aggregation-intensive nature and the precision requirements that are challenging to meet on the memory dies, which are often fabricated using older technology nodes [14]. To further optimize performance, a buffer is integrated into the controller to

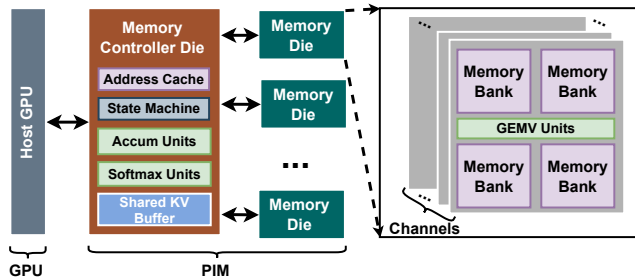


Figure 7: The overall hardware architecture of our proposed ORCHES GPU-PIM collaborative system.

cache frequently accessed data for the host GPU, reducing memory die activations and mitigating interference with near-bank compute units. In addition to the compute units, our design includes an address cache to optimize memory access for the PIM device, because the address mapping is the same for different banks, the address cache is placed in the controller die. More details will be introduced in Sec. 4.4. **Memory Dies** are responsible for both data storage and in-memory computation. Each die contains multiple channels, with each channel comprising several bank groups. Within each bank group are multiple banks, each equipped with dedicated compute units, such as General Matrix-Vector Multiplication (GEMV) units, optimized for parallel execution directly within memory. The GEMV units here are implemented using multiplier-adder trees.

4.2 Technique 1: Adaptive Assignment Enhancing Parallelism

4.2.1 Technique 1A: Offline Parallelization Strategy. As mentioned in Sec. 3.1.1, since multiple branches exist in each reasoning step and the KV query is shared across those branches, both the linear and attention operators can be either compute-bound or memory-bound depending on the specific number of branches configured by users. This behavior differs from standard LLM inference, in which the linear operators are often compute-bound and the attention operators are memory-bound [39]. Hence, existing PIM solutions built on assumptions from standard LLM inference [9, 25] cannot be directly applied to our target LLM reasoning workloads. To bridge this gap, we propose the following parallelization strategies for the operators in TTC-based LLM reasoning, as detailed below:

Linear Operator and Shared Attention Operator: For each linear operator and the attention operator shared across different branches, their arithmetic intensity is proportional to the corresponding batch size (e.g., sequence length or number of branches). Specifically, when the batch size W for a given operator is sufficiently large, the workload becomes compute-bound, indicating that assigning it to the GPU is more efficient than executing it on PIM. Otherwise, when the batch size is small, the workload is memory-bound, and executing it on PIM modules yields higher efficiency. **Unique Attention Operator:** For the attention operator that is unique to each reasoning branch, the batch size is always 1. Therefore, the arithmetic intensity remains fixed at 2, significantly lower than the maximum achievable arithmetic intensity on GPUs (> 500). Therefore, offloading this operator to PIM modules is a more efficient option. It is worth noting that the aforementioned fixed arithmetic intensity value of 2 is based on the unique KV query setting in the standard attention mechanism, used here for simplified explanation. For models employing Grouped Query Attention, the arithmetic intensity depends on group size. Our actual experimental implementation uses the appropriate attention mechanism accordingly.

As a result of the aforementioned analysis on different operators in TTC-based LLM reasoning, the key to determining whether to offload the workload to PIM is the batch size W . Thus, the offline computation partitioning between the GPU and PIM modules is guided by an analytical model that characterizes how latency is affected by key parameters: batch size (W), embedding dimension

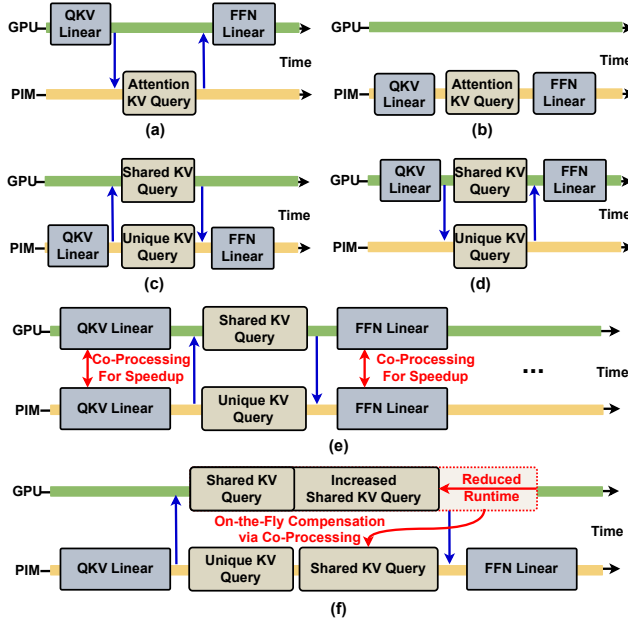


Figure 8: Parallelization strategies for the linear and attention operators in the TTC pipeline. (a) The baseline assignment, where linear operator is always assigned to GPU and the attention operator is always assigned to PIM. (b) The proposed assignment when the total batch size is small. (c) The proposed assignment when the total batch size is in a medium state and the GPU is utilized to help the PIM computation. (d) The proposed assignment when the total batch size is large. (e) The proposed collaborative parallelization strategy. (f) The online scheduling compensation scheme.

(D), compute capability (CC), and memory bandwidth (BW). Specifically, the PIM module has two distinct bandwidths: BW_{PIM_IO} , representing the IO bandwidth of the controller die, and BW_{PIM} , denoting the internal bank-level bandwidth within the memory die. Specifically, taking the linear operator as an example, the latency on GPU and PIM can be represented as:

$$T_{GPU} = \frac{WD^2}{CC_{GPU}} + \frac{2WD + D^2}{BW_{GPU}} \quad (1)$$

$$T_{PIM} = \frac{WD^2}{CC_{PIM}} + \frac{2WD}{BW_{PIM_IO}} + \frac{D^2}{BW_{PIM}} \quad (2)$$

By comparing the latency on GPU and PIM, denoted as T_{GPU} and T_{PIM} , our parallelization strategy is summarized as follows: 1) When the total batch size is small, we assign the linear operator and the entire attention operator (including both the shared and unique KV components) to the PIM modules, as illustrated in Fig. 8(b). 2) When the batch size is moderate and GPU is partially utilized to assist PIM computation, the shared KV query is executed on the GPU, while the linear operator and unique KV query are handled by PIM, as shown in Fig. 8(c). 3) When the batch size is large, both the linear and shared attention operators are executed on the GPU, while the unique KV query is handled by PIM, as shown in Fig. 8(d).

Additionally, to avoid scenarios where only GPU or PIM is used across the entire system, we incorporate a mechanism to evaluate whether utilizing both devices simultaneously is beneficial, as shown in Fig. 8(e). For example, if PIM is initially selected as the primary device, we compare the latency of (i) transferring part of the data to the GPU, executing the operator on the GPU, and transferring the result back to PIM, vs. (ii) running the entire operator segment directly on PIM. If the former is faster, we switch to using the GPU as the primary device for that portion of computation. In particular, we introduce a ratio α to represent the portion of the operator to be executed on the GPU:

$$T_{PIM}(\alpha) = \frac{WD^2(1-\alpha)}{CC_{PIM}} + \frac{WD(2-\alpha)}{BW_{PIM_IO}} + \frac{D^2}{BW_{PIM}} \quad (3)$$

$$T_{GPU}(\alpha) = \frac{WD^2\alpha}{CC_{GPU}} + \frac{WD(1+\alpha) + D^2\alpha}{BW_{GPU}} \quad (4)$$

If the following inequality holds: $T_{PIM} \geq \max(T_{PIM}(\alpha), T_{GPU}(\alpha))$, transferring $\alpha \times 100\%$ of the data to the GPU to execute the operator results in lower latency compared to executing the entire operator on PIM without data transfer. The value of α is determined by solving $T_{GPU}(\alpha) = T_{PIM}(\alpha)$, which minimize $\max(T_{PIM}(\alpha), T_{GPU}(\alpha))$.

The co-processing of GPU and PIM introduces data movement overhead. Since α divides the linear in the output dimension, the communication overhead typically lies in sending the entire FP16 input vector to PIM and collecting the partial FP16 output vector from PIM. The volume of data movement is $WD(2-\alpha)$, which has been considered in the second term of Equation 3. Considering $W \ll D$, D^2 rather than $WD(2-\alpha)$ dominates the latency. Experimental results averaged across all settings in Sec. 5.2 show that data transfer accounts for approximately 8.3% of the total runtime.

Technique 1A: Offline Parallelization Strategy

We propose an offline parallelization strategy for the linear and attention operators in the TTC pipeline. The proposed strategy supports both GPU and PIM devices and integrates seamlessly into TTC workflows deployed on existing hardware systems.

4.2.2 Technique 1B: Online Scheduling Compensation. The aforementioned offline parallelization strategy is designed for operators whose computational workloads remain constant during runtime. However, for the attention operator shared across different reasoning branches, the workload varies at runtime due to the accumulation of shared inputs across branches. To address this challenge, we propose an online scheduling compensation scheme, as illustrated in Fig. 8(f). Considering $Q \cdot K$ as an example, for the increasing compute workload $\sum_i W_i L_i D$, where L_i is the length of a KV cache fragment KV_i (i.e., shared KVs or unique KVs in each reasoning stage), D is the hidden dimension, and W_i is the corresponding batch size (i.e., number of branches), we dynamically compute the value of α before each reasoning step, as L_i increases during the reasoning process. The runtime of PIM and GPU is approximated

by the roofline model:

$$T_{PIM}(\{\alpha_0, \dots, \alpha_i, \dots\}) = \sum_i \frac{W_i L_i D (1 - \alpha_i)}{CC_{PIM}} + \sum_i \frac{L_i D}{BW_{PIM}} \quad (5)$$

$$+ \sum_i \frac{W_i D + W L_i (1 - \alpha_i)}{BW_{PIM_IO}} \quad (6)$$

$$T_{GPU}(\{\alpha_0, \dots, \alpha_i, \dots\}) = \sum_i \frac{W_i L_i D \alpha_i}{CC_{GPU}} + \sum_i \frac{W_i D + W L_i \alpha_i}{BW_{GPU}} \quad (7)$$

This model is employed to quickly initialize an ideal α , which achieves $T_{PIM}(\{\alpha_0, \dots, \alpha_i, \dots\}) = T_{GPU}(\{\alpha_0, \dots, \alpha_i, \dots\})$. Initially, all layers are assigned to the GPU, setting $\alpha_i = 1$. Subsequently, layers are reassigned to the PIM, setting $\alpha_i = 0$, starting from the layer with the lowest W_i to the highest, until T_{PIM} exceeds T_{GPU} . The critical layer t , where the relationship between T_{PIM} and T_{GPU} shifts, is then analyzed. The α values for other layers remain fixed at 0 or 1, while α_t for layer t is treated as a variable. The equation $T_{PIM} = T_{GPU}$ is solved to determine the optimal α_t .

Technique 1A: Offline Parallelization Strategy

We propose an offline parallelization strategy for the linear and attention operators in the TTC pipeline. The proposed strategy supports both GPU and PIM devices and integrates seamlessly into TTC workflows deployed on existing hardware systems.

4.3 Technique 2: Branch Prediction Facilitating Pipelining

4.3.1 Technique 2A: Candidate Verification Predictor. As discussed in Sec. 3.2, branch candidate verification can block the generation in the subsequent step. This verification process is slower than generation, as it typically relies on a larger model (i.e., PRM) to verify the candidates. To mitigate this bottleneck, we propose a branch prediction strategy using a lightweight model, allowing the generation phase to proceed speculatively and overlap with the candidate verification phase. Since the verification phase is primarily compute-bound (i.e., prefilling using a large model on the GPU), the PIM device can be used to perform speculative generation, thereby maximizing utilization of the proposed GPU-PIM system.

As noted in Sec. 3.2, the prefilling speed of the small model is $3\times-5\times$ faster than that of the large model. This speedup enables us to initiate the next step's generation $3\times-5\times$ earlier, reducing the reasoning latency. To preserve generation quality, we draw inspiration from traditional CPU branch prediction: if the speculative prediction is incorrect, we roll back to the correctly selected candidate and regenerate the output. To minimize performance degradation in such cases, we prioritize verification execution and allocate only leftover system resources for speculative generation. Since the GPU is fully occupied with verification, T1 is not activated during speculative generation in T2A. However, T1 is immediately triggered once the verification completes, ensuring full utilization of system resources.

However, this naive use of a smaller model for prediction is not ideal—it may produce inaccurate predictions, triggering frequent rollbacks. In our experiments, prediction accuracy can be as low as 30% for certain models. The root cause lies in the design of the candidate verification mechanism, which depends not only

on the currently generated candidates but also on historical decision scores. Typically, the same model is used to compute scores across all steps. Specifically, at step N , the small model's prediction depends on its own scores from steps 1 to $N-1$. When the small model's earlier predictions are inaccurate, it accumulates unreliable historical context, leading to degraded prediction accuracy. Even when the small model's predictions are correct, its scoring may lack the precision required to support accurate future-step decisions. To address this issue, we propose a history alignment strategy, in which we substitute the small model's historical scores with those from the corresponding steps of the large model, as illustrated in Fig. 9(c). This substitution enriches the predictive context with more accurate and reliable historical data, significantly improving prediction accuracy, as validated in Sec. 5.

Technique 2A: Candidate Verification Predictor

We propose a candidate verification predictor for the TTC pipeline that enables overlapping generation with verification to improve system utilization. It does not affect generation quality.

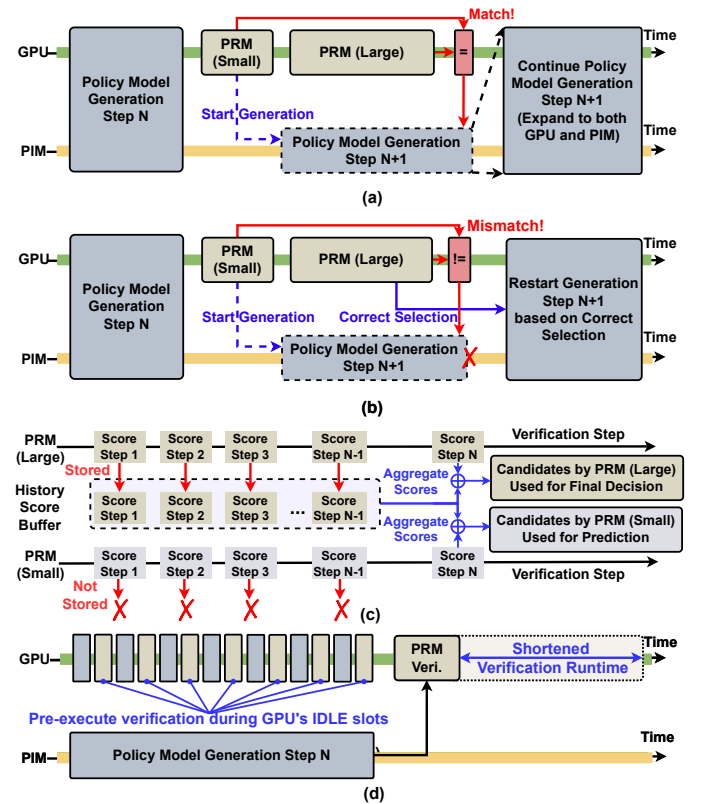


Figure 9: Illustrating the candidate verification predictor and pipelined verification: (a) The workflow when the candidate verification predictor has a successful prediction, (b) The workflow when the candidate verification predictor has a wrong prediction, (c) The workflow when the small model's history scores are replaced with the large model's scores, and (d) the workflow when the pipelined verification is used.

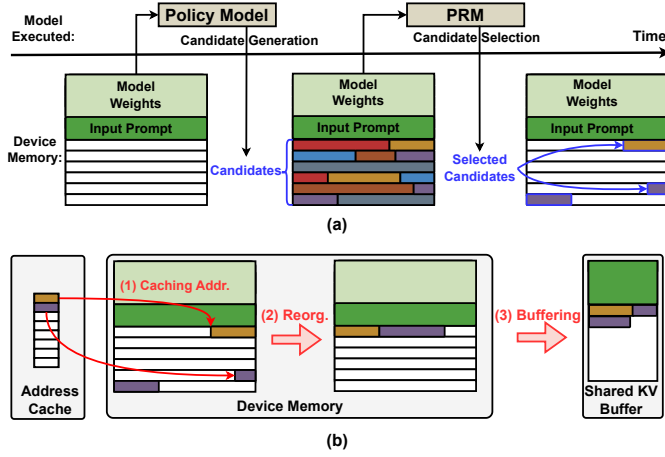


Figure 10: Illustrating the proposed memory structuring in the TTC pipeline for alleviating memory fragmentation. (a) Source of memory sparsity: Candidate verification removes unselected branches, resulting in sparse memory access patterns and memory fragmentation. (b) Overview of the proposed technique: (1) an address caching mechanism for fast indexing, (2) memory reorganization to eliminate sparsity, and (3) a buffering scheme that migrates active branches to the controller die to facilitate efficient GPU access.

4.3.2 Technique 2B: Pipelined Verification. With the aforementioned candidate verification predictor, data dependencies between verification and generation can be alleviated, enabling opportunities to further optimize the verification phase in PRM. Specifically, we propose a pipelined verification approach that initiates candidate verification as soon as the corresponding tokens are generated, rather than waiting for all candidates to be generated beforehand. As illustrated in Fig. 9 (d), once a portion of the candidate tokens is generated, the verification phase is immediately launched if the GPU is idle. To ensure efficient hardware utilization, verification is only triggered when the number of tokens available for prefilling reaches a threshold that ensures sufficient arithmetic intensity to fully utilize the GPU. This pipelined verification is coordinated between the GPU and PIM modules, where pre-verification, i.e., the inference of the small PRM in the proposed candidate verification predictor, is scheduled only when the GPU is idle. This design prioritizes the generation phase and leverages leftover system resources for verification, thereby maximizing overall throughput without interfering with critical generation tasks.

Technique 2B: Pipelined Verification

We propose a pipelined pre-verification strategy for the TTC pipeline that initiates verification as soon as part of the candidate generation is completed, as long as the GPU is idle.

4.4 Technique 3: Memory Structuring Alleviating Fragmentation

To address the memory fragmentation challenge caused by the removal of unselected branches, as illustrated in Fig. 10(a), we propose

a comprehensive memory structuring strategy that integrates three key techniques: address caching, dynamic memory reorganization, and hierarchical buffering.

First, as shown in Fig. 10(b)-(1), we introduce a dedicated memory address cache in the controller die to accelerate access to sparse memory. It maps logical candidate IDs to physical locations, avoiding costly memory traversals when accessing pruned branches. The address cache is introduced to avoid storing pointers directly in DRAM cell lines, which incur high read latency. By using the address cache, two sequential DRAM accesses with data dependency are transformed into a combination of SRAM and DRAM accesses, where the SRAM access is one to two orders of magnitude faster. In terms of overhead, the cache is shared between all banks and only needs to store the location to the beginning of sequences and sequence length (usually less than 1000 datapoints in common benchmarks [8]), so the area overhead remains minimal. The address cache is managed by a state machine in the controller die, which reads pointers from the address cache and then sends instructions to the PIM banks with the processed address. **Second**, as shown in Fig. 10(b)-(2), we implement dynamic memory reorganization. As removing unselected branches introduces memory holes (i.e., unused gaps due to fragmentation), we track fragmentation using a metric β , which is the ratio of *Total Memory Holes* to the *Memory for Reasoning*. When β approaches 1 (i.e., memory is highly fragmented), the system compacts valid blocks into contiguous space to eliminate fragmentation. **Third**, as shown in Fig. 10(b)-(3), the controller’s buffer streamlines reorganization by optimizing memory access. After the QKV is accumulated at the controller die, KV segments are stored in a shared KV buffer, reducing read operations for reorganization. During reorganization, KV segments are written back to the banks to eliminate fragmentation. It avoids PIM-host data transfers, allowing background reorganization. The GPU can also quickly synchronize the KV cache. When decoding is fully offloaded to the PIM, the GPU must fetch the latest KV cache from the PIM.

Technique 3: Memory Structuring Strategy

We propose a three-pronged approach to alleviate memory fragmentation in the TTC acceleration pipeline for (1) fast location lookup, (2) eliminating fragmentation, and (3) optimizing GPU access patterns.

5 Experiments

5.1 Experiment Setting

Hardware Platform Configuration and Baselines: Our baseline GPU platform is the NVIDIA AGX Orin [23]. As for the PIM, we adopt the standardized setup as described in prior work [25], in which each memory bank integrates 16 multipliers and adders. We scale down the total memory capacity of the PIM device to 32GB to better match the constraints of edge environments while still meeting the memory demands of all benchmarks. This configuration results in a total of 2048 memory banks. The off-chip bandwidth in the simulator is configured as 204.8 GB/s to match that of the AGX Orin. In addition to the standalone GPU baseline, we also compare against SOTA GPU-PIM-based LLM inference systems, as described

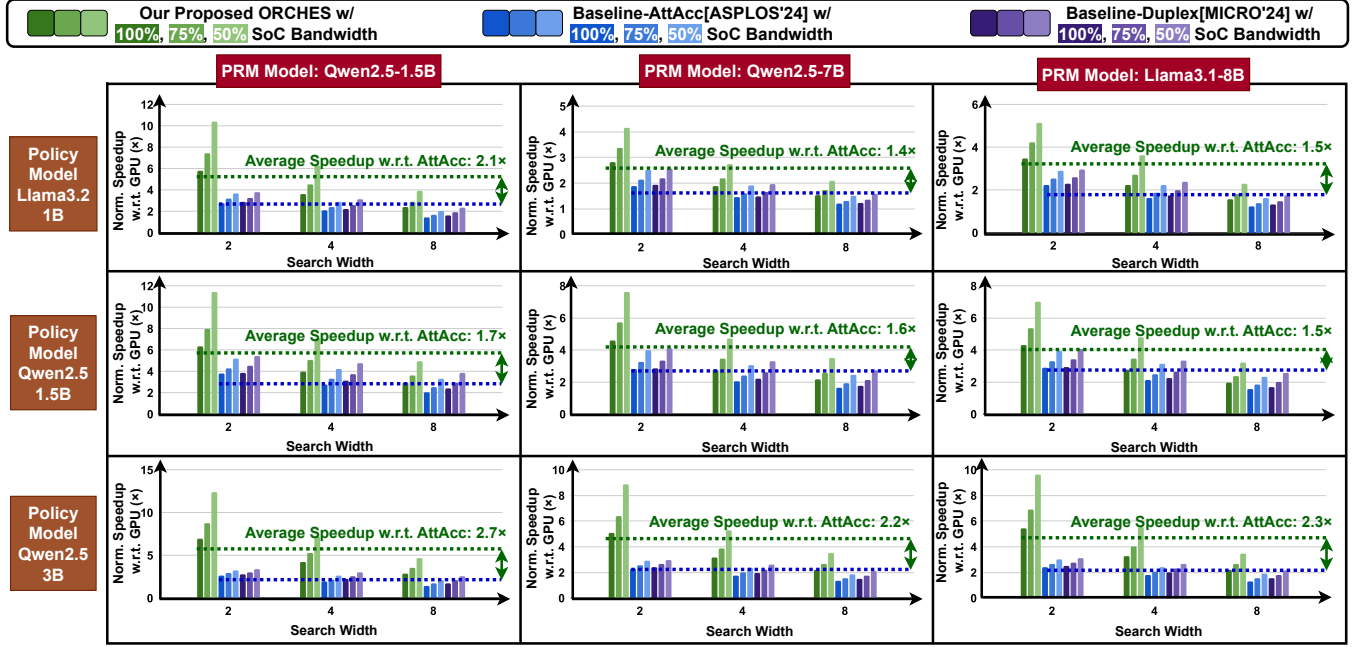


Figure 11: Normalized speedup achieved by the proposed ORCHES system and two baseline devices [25, 40] compared to the baseline GPU platform [23] across different model sizes, search tree widths, and SoC bandwidth on the MATH500 [8] dataset.

in [25, 40]. To further evaluate the performance of our proposed system under varying System-on-Chip (SoC) memory bandwidth constraints, we model the available SoC bandwidth as a variable. Specifically, we test the system under 100%, 75%, and 50% of the total available bandwidth.

Simulator Setup: We build our simulator by extending the open-source AttAcc framework [25, 26]. The simulator leverages a modified version of Ramulator2 [20] to model the memory system. We enhance both the frontend (task scheduling) and the backend (PIM memory system simulation) components to support our proposed TTC-based reasoning pipeline and to reflect the resource constraints of the targeted edge platform. The simulator’s accuracy of GPU modeling and PIM estimating has been validated against real hardware in prior work [25], so we keep the same unit latency and energy consumption as the prior work. To enable system-level energy evaluation, performance counters are implemented in the simulator to count the data volume transferred and the number of different types of computations performed, which are multiplied by the corresponding unit energy values.

Algorithm Pipeline and Dataset: To evaluate our hardware platform, we employ two SOTA algorithm pipelines in TTC-based LLM reasoning: a text-based reasoning pipeline from [18] and a vision-based pipeline from [36]. In the text-based pipeline, the policy model candidates for generation include Llama3.2-1B [33], Qwen2.5-1.5B [37], and Qwen2.5-3B [37], while the PRM models for verification include Qwen2.5-1.5B-PRM-Tuned, Qwen2.5-7B-PRM-Tuned, and Llama3.1-8B-PRM-Tuned. We evaluate all $3 \times 3 = 9$ combinations of these models across 2 ~ 8 branch counts. As reported in [18], these configurations achieve generation quality comparable to or exceeding that of significantly larger models, such as

Llama3.1-405B [33] ($45\times$ larger). We first use the same evaluation dataset as the original work: MATH500 [8], which primarily focuses on math problems. To assess the generality of our design across more diverse use cases, we additionally evaluate it on a coding task dataset, LiveCodeBench [11]. For the vision-based pipeline, we adopt the SOTA approach in [36], which utilizes a fine-tuned Llama-3.2-11B-Vision-Instruct model for both the policy and PRM components. The search tree width is set to 2 and 4. As for the dataset, we use MATHVista [19]. The performance of this setup surpasses closed-source models (e.g., GPT-4o-mini), as well as larger open-source models (e.g., Llama-3.2-90B-Vision-Instruct).

5.2 System Evaluation

For Text-based Tasks: Fig. 11 presents the normalized speedup of the ORCHES system compared to the baseline GPU [23] for text-based tasks. The evaluation covers various model sizes, search tree widths, and available SoC memory bandwidth using the MATH500 dataset [8] and employs the SOTA TTC-based text LLM reasoning pipeline [18]. On average, the proposed ORCHES system achieves a

PRM \ Policy	Llama3.2-1B	Qwen2.5-1.5B	Qwen2.5-3B
Qwen2.5-1.5B	1.96×	2.07×	1.87×
Qwen2.5-7B	3.23×	2.57×	2.14×
Llama3.1-8B	3.4×	2.71×	2.13×

Table 1: Normalized energy efficiency achieved by the proposed ORCHES system compared to the baseline GPU platform [23], averaged across different search tree widths, and question lengths.

Bandwidth \ PRM	Qwen2.5-1.5B	Qwen2.5-7B	Llama3.1-8B
100% Bandwidth	3.85×	3.19×	2.73×
75% Bandwidth	4.98×	3.77×	3.27×
50% Bandwidth	6.93×	5.10×	4.31×

Table 2: Normalized speedup achieved by the proposed ORCHES system compared to the baseline GPU platform [23] on the LiveCodeBench [11] dataset.

	Short QA	Medium QA	Long QA
Width = 2	3.26 ×	3.35 ×	4.85 ×
Width = 4	2.47 ×	2.35 ×	2.32 ×

Table 3: Normalized speedup achieved by the proposed ORCHES system compared to the baseline GPU platform [23] across different search tree widths, and question lengths on the MATHVista [19] dataset.

speedup of 4.16× over the GPU baseline. Tab. 1 summarizes energy efficiency results, indicating an average improvement of 2.45× over the baseline GPU platform. Tab. 2 summarizes the speedup on the coding task, indicating an average speedup of 4.24× over the baseline GPU platform. In addition, we have the following observations: (1) The speedup depends on the search tree width; wider trees generally result in a lower speedup, transitioning workload characteristics from memory-bound to compute-bound. (2) The speedup of the ORCHES system becomes even better as the available bandwidth to the GPU decreases. This is because reduced bandwidth further slows down the decoding process in LLM inference, precisely the stage where PIM devices provide the most benefit. (3) The proposed ORCHES system demonstrates performance improvements across different task types.

For Vision-based Tasks: We evaluate the proposed ORCHES system across varying search tree widths and question lengths on the MATHVista dataset [19], using the SOTA TTC-based vision reasoning pipeline [36]. As shown in Tab. 3, the proposed system achieves an average speedup of 3.10× over the baseline GPU platform. The results also show consistent speedups (i.e., 2.32×-4.85×) across different question lengths.

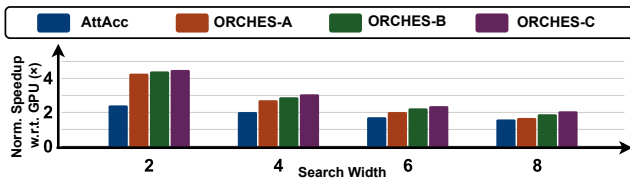


Figure 12: Impact of different scheduling strategies on speedup performance. The evaluated strategies include the baseline GPU platform, a prior work [25], and the proposed ORCHES system (for settings of ORCHES-A, ORCHES-B, and ORCHES-C, please refer to the configurations in Section 5.3). The evaluation is conducted on the MATH500 [8] dataset.

	Llama3.2-1B	Qwen2.5-1.5B	Qwen2.5-3B
Level 1	51.4% → 73.3%	56.1% → 82.4%	61.1% → 79.5%
Level 2	50.7% → 80.1%	56.8% → 82.6%	61.5% → 79.2%
Level 3	53.2% → 82.2%	57.5% → 82.8%	59.9% → 79.5%
Level 4	52.7% → 82.3%	57.7% → 83.1%	59.7% → 79.6%
Level 5	52.6% → 83.0%	57.9% → 83.1%	59.8% → 80.3%

Table 4: Prediction accuracy of selected branches by the PRM with and without (denoted by →) the proposed history alignment mechanism, evaluated across different policy model sizes and question difficulty levels (simplest: Level 1; hardest: Level 5) on the MATH500 [8] dataset.

PRM \ Policy	Llama3.2-1B	Qwen2.5-1.5B	Qwen2.5-3B
Qwen2.5-1.5B	63%	68%	67%
Qwen2.5-7B	64%	71%	65%
Llama3.1-8B	66%	78%	65%

Table 5: Context memory footprint saving of the proposed T3 across different policy and PRM model sizes. All the results are evaluated on the MATH500 [8] dataset.

5.3 Analysis of Technique 1

Adaptive Assignment Enhancing Parallelism. To further evaluate the effectiveness of Technique 1, we conducted an ablation study comparing the speedup of the ORCHES system under different scheduling strategies. Specifically, we compare our system against the baseline GPU and the Attacc scheduling strategy from a prior work [25], across varying search tree widths using a 3B policy model and a 7B PRM model. Technique 2 is turned off for the sake of the ablation study. We consider three configurations of the ORCHES system: 1) **ORCHES-A**, where all computations are offloaded to PIM to favor edge deployment; 2) **ORCHES-B**, where linear layers are adaptively assigned to either GPU or PIM; 3) **ORCHES-C**, which builds upon ORCHES-B by incorporating dynamic compensation mechanisms, enabling the PIM to assist GPU computation when the shared workload becomes substantial. As shown in Fig. 12, the proposed ORCHES system achieves an average 3× speedup over the baseline GPU platform and a 1.5× speedup over prior work [25], which itself achieves a 2× speedup over the baseline. Furthermore, the results indicate that progressively integrating the proposed features yields additional performance improvements.

5.4 Analysis of Technique 2

Candidate Verification Predictor. To evaluate the effectiveness of our proposed candidate verification predictor and its history alignment mechanism, we analyze the predictor’s accuracy across different configurations. Tab. 4 presents the accuracy improvements achieved by our proposed history alignment mechanism in the candidate verification predictor. The results show that without history alignment, the average candidate verification accuracy is approximately 52% across different policy model sizes and question difficulty levels. After applying our history alignment mechanism, the accuracy improves to about 78%, demonstrating a significant enhancement in the system’s ability to select correct reasoning paths. This improvement is consistent across different model sizes and

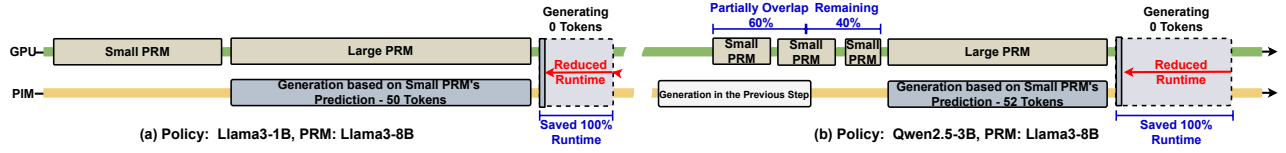


Figure 13: Case studies of the proposed Technique 2, conducted on the MATH500 [8] dataset. The configurations include a 1B and 3B policy model, as well as an 8B large PRM model. The length of all blocks is scaled based on real runtime.

question difficulty levels, indicating the robustness of our approach. We conjecture that the lower prediction accuracy at simpler difficulty levels (e.g., level 1) arises because candidate selection plays a less critical role in the final reasoning outcome for simpler cases. As a result, the predicted candidates may differ more from those ultimately selected by the PRM.

Analysis on Pipelined Candidate Generation and Verification. Our candidate verification predictor enables efficient pipelining between candidate generation and verification phases. To demonstrate the effectiveness of this pipeline strategy in enhancing hardware efficiency, we present two case studies in Fig. 13. In Fig. 13, the "small PRM" denotes the execution of the first 10 layers of the original 8B PRM model, while the "large PRM" comprises the execution of the remaining layers. This partitioning ensures no additional computational overhead. Fig. 13(a) demonstrates the elimination of the latency originally associated with sequential candidate generation. This improvement results from: (1) accurate prediction of candidate verification, and (2) overlapping verification with candidate generation. Fig. 13(b) further enhances pipeline efficiency by pre-executing 60% of the small PRM verification concurrently with generation. This significantly reduces verification latency while fully overlapping with the generation workload.

5.5 Analysis of Technique 3

Memory Structuring Alleviating Fragmentation. With the proposed Technique 3, we are able to save memory footprint by merging the isolated data in the fragment memory. The saved memory corresponds to the context KV cache. During the reasoning process, only the selected branches are executed; unselected branches and their associated context data are reorganized for cleanup. Currently, this reorganization is triggered after 3-5 reasoning steps (i.e., after every 3-5 PRM verification runs). Tab. 5 shows the context memory footprint saving ratio of the proposed Technique 3. Specifically, the proposed Technique 3 can save 65% of context memory footprint on average. However, we did observe that memory reorganization introduces some overhead, primarily due to the additional KV buffer and the runtime costs associated with memory read and write operations. In terms of area, the overhead of the added buffer is 12% under the same hardware implementation settings. For runtime, we evaluate the overhead using the same setup as in the text-based evaluation (Sec. 5.2). Results show that the average runtime overhead is only 0.12%, which is negligible in practice.

5.6 Individual Technique's Contribution

Individual Technique's Contribution to the Overall Speedup: To further evaluate the effectiveness of the individual techniques, we conducted an ablation study comparing the speedup of the

Setting \ PRM	Qwen2.5-1.5B	Qwen2.5-7B	Llama3.1-8B
T1 Only	4.1×	2.9×	3.1×
T2 Only	3.1×	2.8×	2.9×
T1 + T2	4.4×	3.2×	3.4×

Table 6: Impact of different technique settings on speedup performance over the baseline GPU platform. The evaluation is conducted on the MATH500 [8] dataset.

ORCHES system under different configurations. Specifically, we compare our system, configured with only T1, only T2, and both T1 + T2, against a baseline GPU platform, across varying RPM model sizes using a 3B policy model. Tab. 6 reports the average speedups across different question difficulty levels. The results indicate that while each technique individually contributes to performance gains, the combination of both techniques yields the highest speedup.

Individual Technique's Contribution to the Resource Utilization: To provide additional insights into system utilization, we analyzed the utilization rates of both the GPU and PIM components in our system. Using the same experimental setup as in our previous evaluations on the MATH500 [8] dataset, we observed average GPU utilization (including both compute and memory) of 97.9%, 62.2%, and 93.21% for T1 Only, T2 Only, and T1 + T2 settings, respectively. Corresponding PIM utilization was 43.6%, 66.7%, and 61.0%. These results demonstrate that only the combined use of T1 and T2 enables high utilization of both the GPU and the PIM. In contrast, applying either technique in isolation results in high utilization for only one of the two devices.

6 Related Work

LLM Acceleration for Edge Devices. To deploy LLM on edge devices, various software-hardware co-designs have been proposed, including pruning[21, 35, 38] and quantization[4, 10, 15]. They compress the computational load and storage overhead, reducing the latency of LLM inference on edge devices. However, a fundamental challenge remains unresolved: the low compute-to-memory ratio of edge LLM inference, typically described by operation per byte (OP/B). Neither of them alters the OP/B characteristics, which results in limited utilization of GPU or NPU.

Speculative Execution in LLM. Speculative execution in LLM, commonly known as speculative decoding, adapts the draft-then-verify paradigm to accelerate token generation[3, 16, 22, 30, 42]. They employ a lightweight model to speculatively generate multiple tokens and verify the results in parallel with the target LLM. Branch prediction in ORCHES is different from speculative decoding. While speculative decoding speeds up generation by offloading work to a

lightweight model, branch prediction seeks to start decoding earlier by predicting which path the beam search is likely to follow.

Memory Management for KV-Cache. The KV-cache grows and shrinks dynamically during generation. Naively allocating memory based on the maximum possible length can lead to significant memory waste[42]. PageAttention[13] addresses this issue by using a management scheme inspired by the page table in operating systems, which enables dynamic and non-contiguous memory allocation for the KV-cache. However, the non-contiguous memory allocation impacts the latency of memory access, since the only contiguous memory access could utilize the burst mechanism[1]. In contrast, T3 of ORCHES achieves both the elimination of memory waste and the contiguous storage of kv-cache data.

7 Conclusion

We propose, design, and evaluate a system, which aims to enable the deployment of TTC-based LLM reasoning on edge devices. Experimental results demonstrate that ORCHES achieves a 4.16 $\times$ and 3.10 $\times$ average speedup over SOTA GPU implementations on representative text- and vision-based reasoning tasks, respectively.

Acknowledgments

This article is based upon work supported by the National Science Foundation (NSF) (Award IDs: 1937592, 2048183, 2016727, and 2434166), the Department of Health and Human Services Advanced Research Projects Agency for Health (ARPA-H) under Award Number AY1AX 000003 and Agreement Number 140D042490003, and CoCoSys, one of the seven centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by DARPA. The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied of the Advanced Research Projects Agency Health or the U.S. Government.

References

- [1] Mikhail Asiaty and Paolo Ienne. 2019. Dynaburst: Dynamically assembling dram bursts over a multitude of random accesses. In *2019 29th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 254–262.
- [2] Jing Bi, Junjia Guo, Susan Liang, Guangyu Sun, Luchuan Song, Yunlong Tang, Jinxi He, Jiarui Wu, Ali Vosoughi, Chen Chen, and Chenliang Xu. 2025. VERIFY: A Benchmark of Visual Explanation and Reasoning for Investigating Multimodal Reasoning Fidelity. *arXiv preprint arXiv:2503.11557* (2025).
- [3] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318* (2023).
- [4] Yuzong Chen, Ahmed F AbouElhamayed, Xilai Dai, Yang Wang, Marta Andronic, George A Constantinides, and Mohamed S Abdelfattah. 2025. Bitmod: Bit-serial mixture-of-datatype llm acceleration. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1082–1097.
- [5] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. Prime: A novel processing-in-memory architecture for neural network computation in rram-based main memory. *ACM SIGARCH Computer Architecture News* 44, 3 (2016), 27–39.
- [6] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948 [cs.CL]* <https://arxiv.org/abs/2501.12948>
- [7] Hao Fei, Shengqiong Wu, Wei Ji, Hanwang Zhang, Meishan Zhang, Mong Li Lee, and Wynne Hsu. 2024. Video-of-thought: step-by-step video reasoning from perception to cognition. In *Proceedings of the 41st International Conference on Machine Learning*. 13109–13125.
- [8] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.
- [9] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. 2024. Neupims: Npu-pim heterogeneous acceleration for batched llm inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 722–737.
- [10] Weiming Hu, Haoyan Zhang, Cong Guo, Yu Feng, Renyang Guan, Zhendong Hua, Zihan Liu, Yue Guan, Minyi Guo, and Jingwen Leng. 2025. M-ANT: Efficient Low-bit Group Quantization for LLMs via Mathematically Adaptive Numerical Type. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1112–1126.
- [11] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. [n. d.]. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *The Thirteenth International Conference on Learning Representations*.
- [12] Yoongu Kim, Weikun Yang, and Onur Mutlu. 2015. Ramulator: A fast and extensible DRAM simulator. *IEEE Computer Architecture Letters* 15, 1 (2015), 45–49.
- [13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [14] Young-Cheon Kwon, Suk Han Lee, Jaehoon Lee, Sang-Hyuk Kwon, Je Min Ryu, Jong-Pil Son, Seongil O, Hak-Soo Yu, Haesuk Lee, Soo Young Kim, Youngmin Cho, Jin Guk Kim, Jongyoon Choi, Hyun-Sung Shin, Jin Kim, BengSeng Phuah, HyoungMin Kim, Myeong Jun Song, Ahn Choi, Daeho Kim, SooYoung Kim, EunBong Kim, David Wang, Shinhaeng Kang, Yuhwan Ro, Seungwoo Seo, JoonHo Song, Jaeyoun Youn, Kyomin Sohn, and Nam Sung Kim. 2021. 25.4 a 20nm 6gb function-in-memory dram, based on hbm2 with a 1.2 tflops programmable computing unit using bank-level parallelism, for machine learning applications. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 64. IEEE, 350–352.
- [15] Jungi Lee, Wonbeom Lee, and Jaewoong Sim. 2024. Tender: Accelerating large language models via tensor decomposition and runtime quantization. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1048–1062.
- [16] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*. PMLR, 19274–19286.
- [17] Weitao Li, Pengfei Xu, Yang Zhao, Haitong Li, Yuan Xie, and Yingyan Lin. 2020. Timely: Pushing data movements and interfaces in pim accelerators towards local and in time domain. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 832–845.
- [18] Runze Liu, Junqi Gao, Jian Zhao, Kaiyan Zhang, Xiu Li, Biqing Qi, Wanli Ouyang, and Bowen Zhou. 2025. Can 1B LLM Surpass 405B LLM? Rethinking Compute-Optimal Test-Time Scaling. *arXiv preprint arXiv:2502.06703* (2025).
- [19] Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. 2023. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. *arXiv preprint arXiv:2310.02255* (2023).
- [20] Haocong Luo, Yahya Can Tuğrul, F Nisa Bostancı, Ataberk Olgun, A Giray Yağlıkcı, and Onur Mutlu. 2023. Ramulator 2.0: A modern, modular, and extensible dram simulator. *IEEE Computer Architecture Letters* 23, 1 (2023), 112–116.
- [21] Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems* 36 (2023), 21702–21720.
- [22] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Rae Ying Yee Wong, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. 2023. Specinfer: Accelerating generative llm serving with speculative inference and token tree verification. *arXiv preprint arXiv:2305.09781*, 1, 2 (2023), 4.
- [23] NVIDIA. [n. d.]. Jetson Orin for Next-Gen Robotics | NVIDIA. <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>. (Accessed on 04/02/2024).
- [24] OpenAI. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [25] Jaehyun Park, Jaewan Choi, Kwanhee Kyung, Michael Jaemin Kim, Yongsuk Kwon, Nam Sung Kim, and Jung Ho Ahn. 2024. AttAcc! Unleashing the power of PIM for batched transformer-based generative model inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 103–119.
- [26] scale.snu. [n. d.]. Simulator for AttAcc. https://github.com/scale-snu/attacc_simulator. (Accessed on 04/02/2024).
- [27] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramanian, John Paul Strachan, Miao Hu, R Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars. *ACM SIGARCH Computer Architecture News* 44, 3 (2016), 14–26.
- [28] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314* (2024).

- [29] Zhihong Sun, Chen Lyu, Bolun Li, Yao Wan, Hongyu Zhang, Ge Li, and Zhi Jin. 2024. Enhancing Code Generation Performance of Smaller Models by Distilling the Reasoning Ability of LLMs. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*. 5878–5895.
- [30] Ziteng Sun, Ananda Theertha Suresh, Jae Hun Ro, Ahmad Beirami, Himanshu Jain, and Felix Yu. 2023. Spectr: Fast speculative decoding via optimal transport. *Advances in Neural Information Processing Systems* 36 (2023), 30222–30242.
- [31] Llama Team. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [32] Llama Team. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [33] Llama Team. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [34] Yi Wang, Weixuan Chen, Jing Yang, and Tao Li. 2018. Exploiting parallelism for CNN applications on 3D stacked processing-in-memory architecture. *IEEE Transactions on Parallel and Distributed Systems* 30, 3 (2018), 589–600.
- [35] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453* (2023).
- [36] Guowei Xu, Peng Jin, Li Hao, Yibing Song, Lichao Sun, and Li Yuan. 2024. Llava-o1: Let vision language models reason step-by-step. *arXiv preprint arXiv:2411.10440* (2024).
- [37] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115* (2024).
- [38] Shang Yang, Junxian Guo, Haotian Tang, Qinghao Hu, Guangxuan Xiao, Jiaming Tang, Yujun Lin, Zhijian Liu, Yao Lu, and Song Han. 2025. Lserve: Efficient long-sequence llm serving with unified sparse attention. *arXiv preprint arXiv:2502.14866* (2025).
- [39] Zhihang Yuan, Yuzhang Shang, Yang Zhou, Zhen Dong, Zhe Zhou, Chenhao Xue, Bingzhe Wu, Zhikai Li, Qingyi Gu, Yong Jae Lee, Yan Yan, Beidi Chen, Guangyu Sun, and Kurt Keutzer. 2024. Llm inference unveiled: Survey and roofline model insights. *arXiv preprint arXiv:2402.16363* (2024).
- [40] Sungmin Yun, Kwanhee Kyung, Juhwan Cho, Jaewan Choi, Jongmin Kim, Byeongho Kim, Sukhan Lee, Kyomin Sohn, and Jung Ho Ahn. 2024. Duplex: A Device for Large Language Models with Mixture of Experts, Grouped Query Attention, and Continuous Batching. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1429–1443.
- [41] Yang Katie Zhao, Shang Wu, Jingqun Zhang, Sixu Li, Chaojian Li, and Yingyan Celine Lin. 2023. Instant-nerf: Instant on-device neural radiance field training via algorithm-accelerator co-designed near-memory processing. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [42] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, et al. 2024. A survey on efficient inference for large language models. *arXiv preprint arXiv:2404.14294* (2024).